

# State Management

## INFO

### Lesinhoud

In deze les leer je wat state management is, waarom het belangrijk is in frontend development, en hoe je het kan implementeren in Vue.js met behulp van Pinia.

### Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.
- Een single-page frontend applicatie ontwikkelen met Vue.js.
- Zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

## ▼ Inhoud

- [Wat is State Management?](#)
- [Pinia toevoegen](#)
- [Pinia gebruiken](#)
  - [storeToRefs](#)
- [Opslaan van data](#)
- [Conclusie](#)

- [Meer leren](#)

---

## Wat is State Management?

In een applicatie is **state** de data die je gebruikt om de huidige status van de applicatie weer te geven. Dit kan bijvoorbeeld de ingelogde gebruiker zijn, de inhoud van een winkelwagen, voorkeuren van de gebruiker, data die getoond wordt op de pagina ...

Naarmate een applicatie groter en complexer wordt, kan het beheren van deze **state** uitdagend worden. Verschillende componenten moeten mogelijk dezelfde data delen en bijwerken, wat kan leiden tot problemen zoals:

- **Data Inconsistency:** Verschillende componenten kunnen verschillende versies van dezelfde data hebben.
- **Tight Coupling:** Componenten worden sterk afhankelijk van elkaar, wat het moeilijk maakt om ze afzonderlijk te testen of hergebruiken.
- **Complex Data Flow:** Het wordt moeilijk om te volgen hoe data door de applicatie stroomt en waar wijzigingen plaatsvinden.

State management libraries, zoals **Pinia**, bieden een gestructureerde manier om de **state** van een applicatie te beheren. Ze centraliseren de **state** op één plaats, waardoor het gemakkelijker wordt om data te delen tussen componenten, wijzigingen bij te houden en de algehele architectuur van de applicatie te verbeteren.

**Pinia** is een moderne state management library voor Vue.js die ontworpen is om eenvoudiger en intuïtiever te zijn dan zijn voorganger, **Vuex**. Via **Pinia** kunnen er **stores** aangemaakt worden die de **state**, **getters** en **actions** bevatten. Waarbij **state** de data is, **getters** afgeleide data teruggeven (zoals **computed properties** in Vue) en **actions** zijn methodes om de **state** te wijzigen.

## Pinia toevoegen

Om **Pinia** te installeren selecteer je tijdens de setup van het Vue-project de optie **Pinia**. Hierdoor installeer en configureer je **Pinia** automatisch in het project.

```
npm create vue@latest my-store-example
```

sh

```
Vue.js – The Progressive JavaScript Framework
|
|
| ♦ Select features to include in your project: (↑/↓ to
| navigate, space to select, a to toggle all, enter to
| confirm)
|   ☐ TypeScript
|   ☐ JSX Support
|   ☒ Router (SPA development)
|   ☒ Pinia (state management)
|   ☐ Vitest (unit testing)
|   ☐ End-to-End Testing
|   ☒ Linter (error prevention)
|   ☒ Prettier (code formatting)
```

sh

Vink geen experimentele functies aan. En kies ervoor om de voorbeeldcode niet te skippen.

### ▼ Prettier semicolons - **npm install** en **npm run format**

Denk eraan om de **.prettierrc.json** te wijzigen zodat er wel semicolons gebruikt worden.

```
{
  "$schema": "https://json.schemastore.org/prettierrc",
  "semi": true,
  "singleQuote": true,
```

json

```
"printWidth": 100
}
```

Navigeer in de terminal naar de map van het project en installeer de dependencies:

```
npm install
```

sh

Vervolgens formatteer je de code met:

```
npm run format
```

sh

Start de development server met:

```
npm run dev
```

sh

---

## Pinia gebruiken

In de map `src/stores` vind je een voorbeeldstore `counter.js`. Bekijk het bestand.

```
import { defineStore } from "pinia";
import { computed, ref } from "vue";

export const useCounterStore = defineStore("counter", ()
=> {
  const count = ref(0);

  const doubleCount = computed(() => count.value * 2);

  function increment() {
    count.value++;
  }
}
```

js

```
    return { count, doubleCount, increment };
  });
```

Deze store bevat:

- **state**: de **count** variabele die de huidige tellerwaarde bijhoudt.
  - Merk op: in de Composition API definieer je **state** met **ref** of **reactive**.
    - Je gebruikt **ref** voor primitieve waarden zoals nummers, strings en booleans.
    - Je gebruikt **reactive** voor objecten en arrays.
- **getter**: de **doubleCount** computed property die de dubbele waarde van **count** teruggeeft.
- **action**: de **increment** -functie die de **count** verhoogt met 1.

```
<script setup>
import { useCounterStore } from "@stores/counter";

const counterStore = useCounterStore();
</script>

<template>
  <div>
    <p>Count: {{ counterStore.count }}</p>
    <p>Double Count: {{ counterStore.doubleCount }}</p>
    <button
      @click="counterStore.increment">Increment</button>
    </div>
  </template>
```

vue

Merk op dat je de **count** verhoogt via de **action** **increment**, en dat je de **getter** **doubleCount** gebruikt om de verdubbelde waarde weer te geven zonder de originele **state** te wijzigen. Merk ook op dat de data

reactief is: wanneer `count` verandert (door op Increment te klikken), werkt het systeem `doubleCount` automatisch bij in de UI.

### Niet doen - Options API

In dit opleidingsonderdeel werk je met de Composition API. De Options API is de oudere manier om Vue-componenten te schrijven en stores te definiëren. Je gebruikt de Options API in stores en componenten niet in dit opleidingsonderdeel.

```
import { defineStore } from "pinia";

export const useCounterStore = defineStore("counter", {
  state: () => ({
    count: 0,
  }),
  getters: {
    doubleCount: (state) => state.count * 2,
  },
  actions: {
    increment() {
      this.count++;
    },
  },
});
```

js

### Wel doen - Composition API

Gebruik altijd de Composition API om de stores en componenten te schrijven. Dit zorgt voor een consistentere codebase en maakt het gemakkelijker om moderne Vue-functies te gebruiken.

```
import { defineStore } from "pinia";
import { computed, ref } from "vue";

export const useCounterStore = defineStore("counter", () => {
  // State
  const count = ref(0);

  // Getter
```

js

```

const doubleCount = computed(() => count.value * 2);

// Action
function increment() {
  count.value++;
}

return { count, doubleCount, increment };
});

```

## storeToRefs

Wanneer je een store in een component gebruikt, kan je de `storeToRefs` -functie gebruiken om de `state` en `getters` van de store via `destructuring` in afzonderlijke reactieve variabelen te plaatsen. Ook `methods` kunnen via `destructuring` als aparte functies gebruikt worden.

```

<script setup>
import { storeToRefs } from "pinia";
import { useCounterStore } from "@/stores/counter";

const counterStore = useCounterStore();
// Destructure via storeToRefs om reactiviteit te
// behouden
const { count, doubleCount } = storeToRefs(counterStore);
// Destructure methodes rechtstreeks
const { increment } = counterStore;
</script>

<template>
<div>
  <p>Count: {{ count }}</p>
  <p>Double Count: {{ doubleCount }}</p>
  <button @click="increment">Increment</button>

```

vue

```
</div>  
</template>
```

Beide manieren van werken (rechtstreeks aanroepen op de store, of via `destructuring`) zijn correct. In de praktijk gebruik je consistent één van de twee manieren, afhankelijk van de beslissing binnen een team.

Voor dit opleidingsonderdeel mag je kiezen welke manier je gebruikt. Per project werk je consistent met de gekozen aanpak.

---

## Opslaan van data

Stel dat je de state van de applicatie, in dit geval de `count`, tussen het herladen van de pagina wilt bewaren. Dan voeg je bijvoorbeeld in de `method` logica toe om de `count` in `localStorage` op te slaan telkens wanneer deze verandert. Vervolgens voeg je een methode toe om de `count` vanuit `localStorage` te initialiseren wanneer je de store aanmaakt.

```
import { defineStore } from "pinia";  
import { computed, ref } from "vue";  
  
export const useCounterStore = defineStore("counter", ()  
=> {  
  const count = ref(0);  
  
  // Initialiseer count vanuit localStorage  
  function initialize() {  
    const savedCount = localStorage.getItem("count");  
    if (savedCount !== null) {  
      // localStorage slaat alles op als string, zet om  
      // naar een getal  
      count.value = parseInt(savedCount, 10);  
    }  
  }  
}
```

js



```

    }

    const doubleCount = computed(() => count.value * 2);

    function increment() {
      count.value++;
      // Sla de nieuwe count op in localStorage
      localStorage.setItem("count",
count.value.toString());
    }

    // Roep initialize aan bij het aanmaken van de store
    initialize();

    return { count, doubleCount, increment };
  });

```

Stel dat er later een backend API beschikbaar is waarmee je de `count` kan opslaan en ophalen uit een database, i.p.v. `localStorage`. Dan pas je de logica in de `initialize` - en `increment` -methodes aan om deze API in plaats van `localStorage` te gebruiken, zonder dat je de componenten die de store gebruiken hoeft aan te passen.

---

## Conclusie

Een `store` in Pinia is een centrale plaats om de `state`, `getters` en `actions` rondom specifieke data te beheren. Dit maakt het eenvoudiger om data te delen tussen componenten, wijzigingen bij te houden en de algehele architectuur van de applicatie te verbeteren.

Bijvoorbeeld in een applicatie waarin studenten beheerd worden, kan je een `studentStore` maken die de lijst van studenten ( `state` ), het aantal studenten ( `getter` ) en methodes om studenten toe te voegen, te wijzigen of te verwijderen ( `actions` ) bevat. In diezelfde applicatie kan er

een andere store zijn, bijvoorbeeld een `courseStore`, waarmee alle data rondom cursussen beheerd wordt.

---

## Meer leren

Bekijk de [officiële documentatie van Pinia](#) om meer te leren over geavanceerde concepten zoals:

---

Previous page  
[Oefeningen](#)

Next page  
[Oefeningen](#)